Lab-16.3

Name: D.Suraj
2303A53006
Batch:46

Lab 16 – Database Design and Queries: Schema Design and SQL Generation

Task 1 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

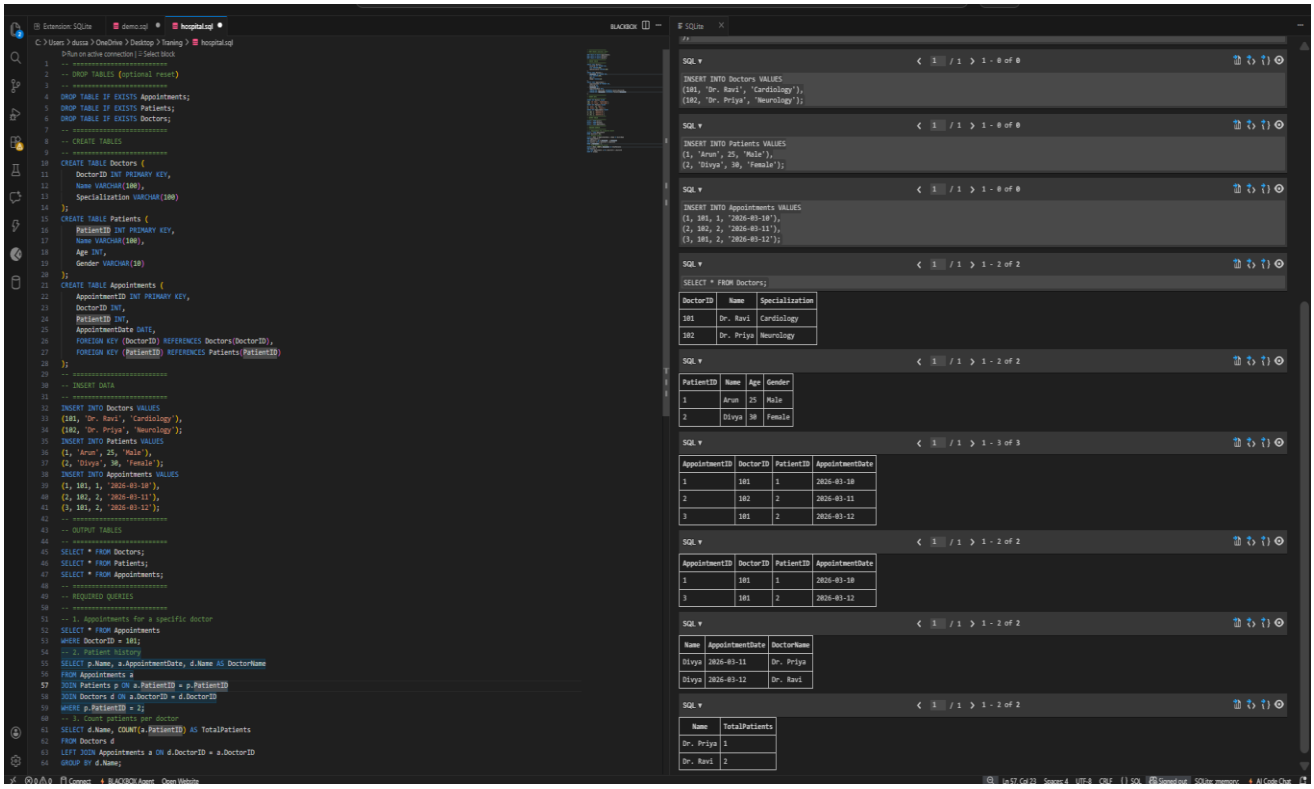
- Tables: Doctors, Patients, Appointments.
- Use AI to define constraints (unique IDs, valid dates).

Generate queries:

- List all appointments for a specific doctor.
- Retrieve patient history by patient ID.
- Count total patients treated by each doctor.

Expected Output:

- Normalized schema and SQL queries with joins.



Execution Result Explanation

- **The database tables Doctors, Patients, and Appointments were successfully created.**
- **Sample data was inserted into all tables.**
- **The output shows:**
  - **Doctors Table with doctor details**
  - **Patients Table with patient records**
  - **Appointments Table showing relationships between doctors and patients**
- **SQL queries were executed successfully:**
  - **Retrieved appointments for a specific doctor**
  - **Displayed patient history using JOIN**
  - **Counted total patients treated by each doctor using aggregation**

Task 2 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

• Tables: Users, Products, Orders, OrderDetails.

• Generate queries to:

• Retrieve all orders by a user.

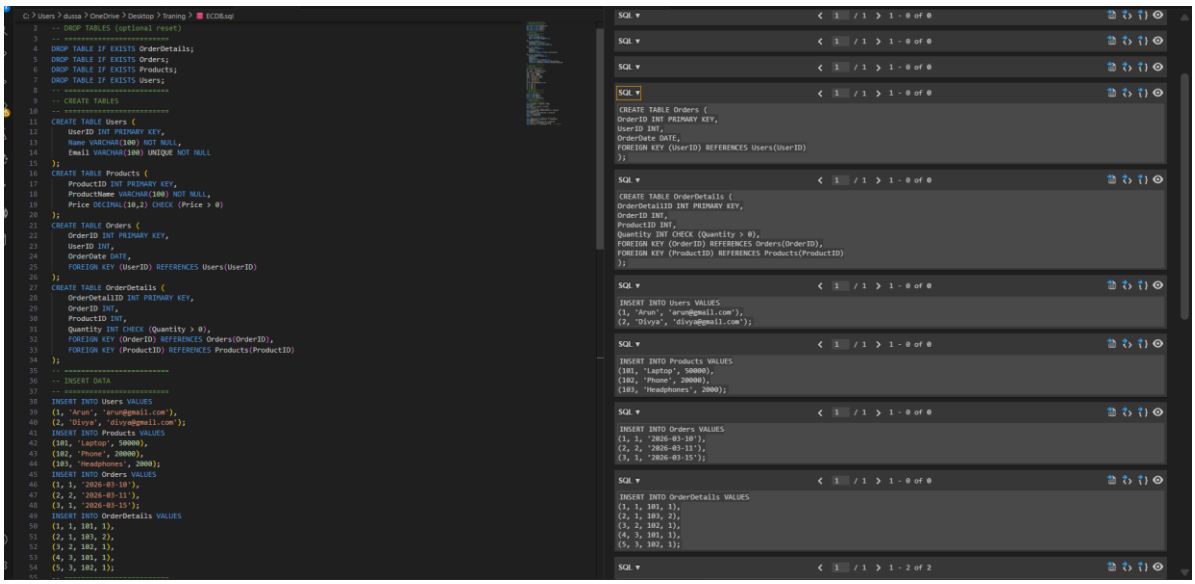
• Find the most purchased product.

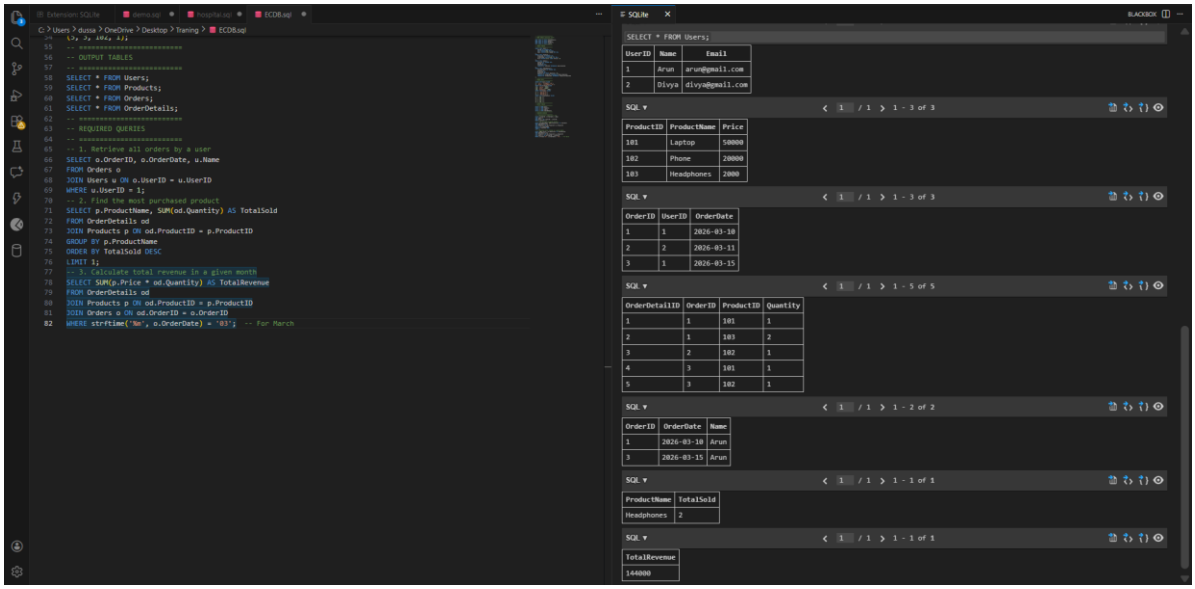
• Calculate total revenue in a given month.

• AI should also suggest normalization improvements and query optimization.

Expected Output:

• Complete schema with relationships + SQL queries for analytics.





Normalization & Improvements

- Data is split into **Users, Products, Orders, OrderDetails** (3NF).
- Avoids redundancy (no repeated product/user data).
- OrderDetails handles **many-to-many relationship**.

Explanation:

The E-commerce database is designed using four tables: Users, Products, Orders, and OrderDetails.
Primary keys and foreign keys are used to maintain relationships and ensure data integrity.
The schema is normalized to eliminate redundancy and organize data efficiently.
SQL queries with JOIN and aggregation functions are used to retrieve user orders, identify the most purchased product, and calculate total revenue.
Query performance is improved using indexing and optimized joins instead of subqueries.

Task 3 – Library Database

Task:

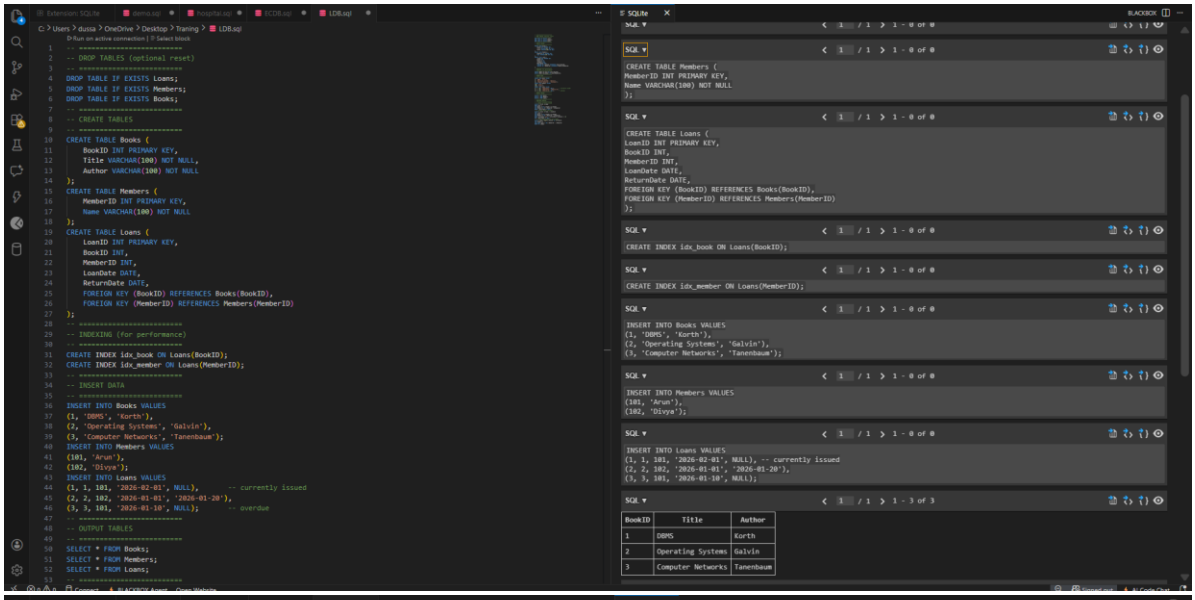
Design schema for a Library Management System.

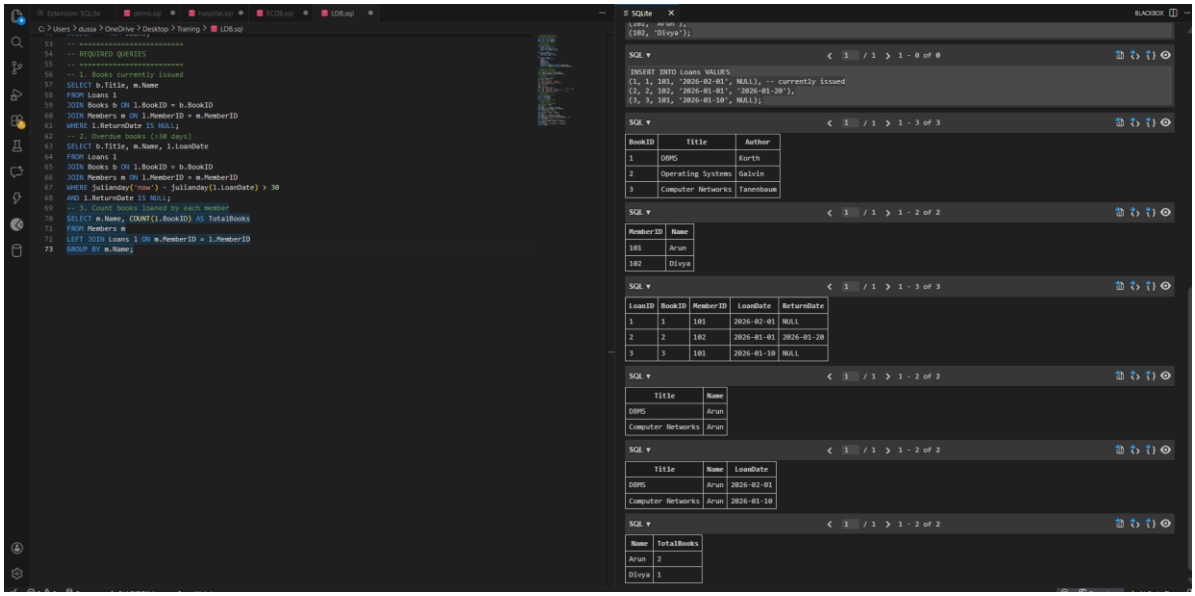
Instructions:

• Tables: Books, Members, Loans.
• Use AI to suggest indexing strategy for faster queries.
• **Generate queries:**
• Retrieve all books currently issued.
• Find overdue books (loan date > 30 days).
• Count number of books loaned by each member.

Expected Output:

• Schema + SQL queries demonstrating joins and conditions.





Explanation:

The Library Management Database is designed using Books, Members, and Loans tables.
Primary and foreign keys maintain relationships and ensure data integrity.
Indexing is applied on Loan table columns to improve query performance.
SQL queries using JOIN and conditions retrieve issued books, overdue books, and count loans per member.
The schema is normalized to avoid redundancy and improve efficiency.

Task 4: Optimize Queries

Instructions:

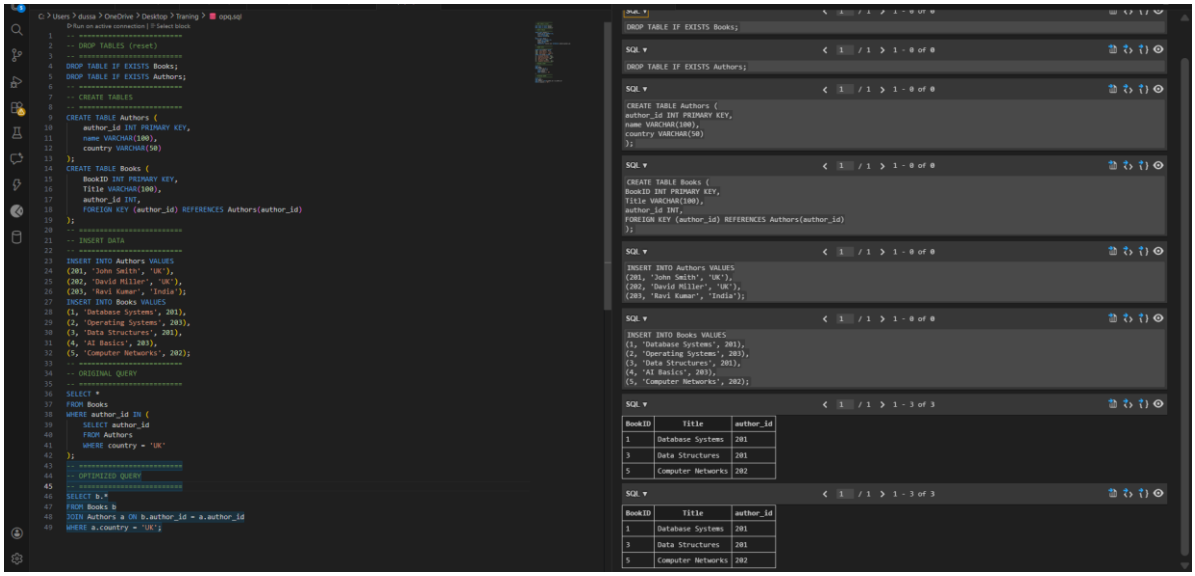
- Use AI tools to analyze query efficiency.
- Optimize inefficient queries using joins, indexes, or reducing subqueries.
- Test optimized queries for correctness.

Starter Code Example:

-- Original query
SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM Authors WHERE country='UK');
-- Optimized query
SELECT b.*
FROM Books b
JOIN Authors a ON b.author_id = a.author_id
WHERE a.country = 'UK';

Expected Output:

• Both queries return the same result: all books written by UK authors.
• Optimized query performs faster for larger datasets



Explanation

The original query uses a subquery to filter books based on authors from the UK, which can be less efficient for large datasets.
The optimized query replaces the subquery with a JOIN, allowing faster data retrieval by directly combining both tables.
JOIN operations are more efficient as they utilize indexing and reduce repeated scanning of tables.
Both queries return the same result, but the optimized query improves performance and scalability.
Thus, query optimization helps in reducing execution time and improving database efficiency.

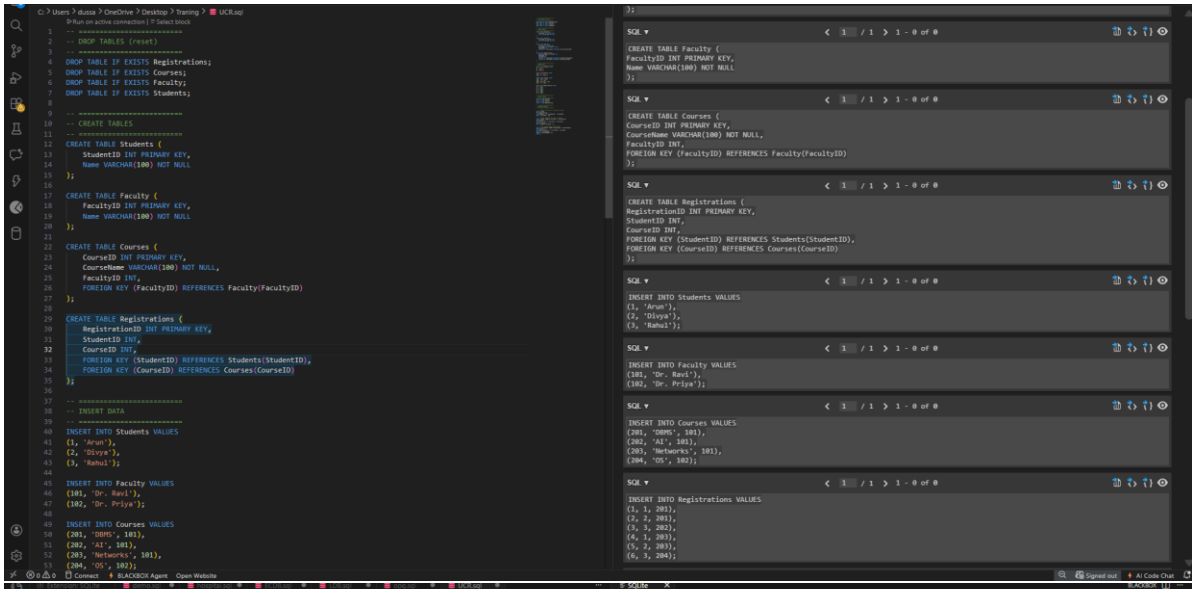
Task 5: University Course Registration

Scenario:

SR University needs a course registration system for students enrolling in different courses under faculty supervision.

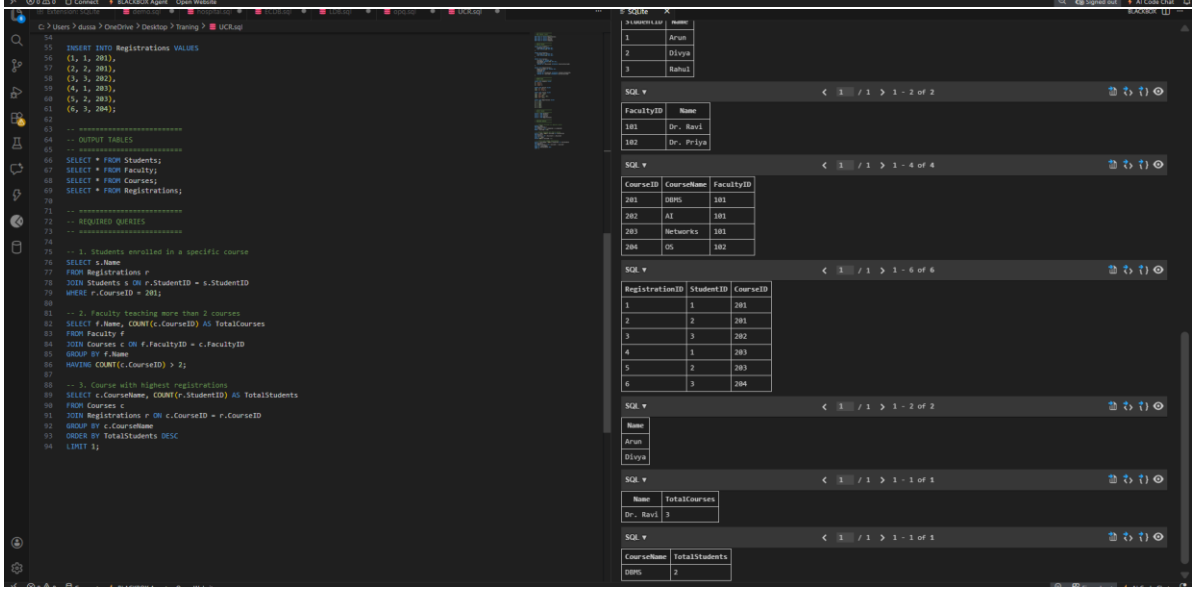
• Use Copilot to design schema tables: Students, Courses, Faculty, and Registrations.
• Define relationships:
• Each student can register for multiple courses.

- Each course is taught by a faculty member.
- **Write SQL queries to:**
- List all students enrolled in a specific course.
- Find faculty members teaching more than 2 courses.
- Retrieve courses with the highest number of registrations.



```
-- ========================
-- DROP TABLES (reset)
-- ========================
DROP TABLE IF EXISTS Registrations;
DROP TABLE IF EXISTS Courses;
DROP TABLE IF EXISTS Faculty;
DROP TABLE IF EXISTS Students;

-- ========================
-- CREATE TABLES
-- ========================
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL
);

CREATE TABLE Faculty (
    FacultyID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL
);

CREATE TABLE Courses (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(100) NOT NULL,
    FacultyID INT,
    FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)
);

CREATE TABLE Registrations (
    RegistrationID INT PRIMARY KEY,
    StudentID INT,
    CourseID INT,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);

-- ========================
-- INSERT DATA
-- ========================
INSERT INTO Students VALUES
(1, 'Arun'),
(2, 'Divya'),
(3, 'Rahul');

INSERT INTO Faculty VALUES
(101, 'Dr. Ravi'),
(102, 'Dr. Priya');

INSERT INTO Courses VALUES
(201, 'DBMS', 101),
(202, 'AI', 101),
(203, 'Networks', 101),
(204, 'OS', 102);
```

```
CREATE TABLE Faculty (
FacultyID INT PRIMARY KEY,
Name VARCHAR(100) NOT NULL
);

CREATE TABLE Courses (
CourseID INT PRIMARY KEY,
CourseName VARCHAR(100) NOT NULL,
FacultyID INT,
FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)
);

CREATE TABLE Registrations (
RegistrationID INT PRIMARY KEY,
StudentID INT,
CourseID INT,
FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
);

INSERT INTO Students VALUES
(1, 'Arun'),
(2, 'Divya'),
(3, 'Rahul');

INSERT INTO Faculty VALUES
(101, 'Dr. Ravi'),
(102, 'Dr. Priya');

INSERT INTO Courses VALUES
(201, 'DBMS', 101),
(202, 'AI', 101),
(203, 'Networks', 101),
(204, 'OS', 102);

INSERT INTO Registrations VALUES
(1, 1, 201),
(2, 2, 201),
(3, 3, 202),
(4, 1, 203),
(5, 2, 203),
(6, 3, 204);
```



```
INSERT INTO Registrations VALUES
(1, 1, 201),
(2, 2, 201),
(3, 3, 202),
(4, 1, 203),
(5, 2, 203),
(6, 3, 204);

-- ========================
-- OUTPUT TABLES
-- ========================
SELECT * FROM Students;
SELECT * FROM Faculty;
SELECT * FROM Courses;
SELECT * FROM Registrations;

-- ========================
-- REQUIRED QUERIES
-- ========================

-- 1. Students enrolled in a specific course
SELECT s.Name
FROM Registrations r
JOIN Students s ON r.StudentID = s.StudentID
WHERE r.CourseID = 201;

-- 2. Faculty teaching more than 2 courses
SELECT f.Name, COUNT(c.CourseID) AS TotalCourses
FROM Faculty f
JOIN Courses c ON f.FacultyID = c.FacultyID
GROUP BY f.Name
HAVING COUNT(c.CourseID) > 2;

-- 3. Course with highest registrations
SELECT c.CourseName, COUNT(r.StudentID) AS TotalStudents
FROM Courses c
JOIN Registrations r ON c.CourseID = r.CourseID
GROUP BY c.CourseName
ORDER BY TotalStudents DESC
LIMIT 1;
```

1	Arun
2	Divya
3	Rahul

FacultyID	Name
101	Dr. Ravi
102	Dr. Priya

CourseID	CourseName	FacultyID
201	DBMS	101
202	AI	101
203	Networks	101
204	OS	102

RegistrationID	StudentID	CourseID
1	1	201
2	2	201
3	3	202
4	1	203
5	2	203
6	3	204

Name
Arun
Divya

Name	TotalCourses
Dr. Ravi	3

CourseName	TotalStudents
DBMS	2

Explanation

The University Course Registration system is designed using Students, Faculty, Courses, and Registrations tables.

Relationships are maintained using primary and foreign keys to ensure data integrity.

Each student can enroll in multiple courses, and each course is assigned to a faculty member.

SQL queries using JOIN and aggregation are used to retrieve enrolled students, identify faculty teaching multiple courses, and find the most popular course.

The schema is normalized to reduce redundancy and improve efficiency.